

C语言与程序设计

The C and Programming

第3章 基本的标准输入与输出

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院

- ◆ 输入是将外部数据输送到计算机中的过程;
- ◆ 输出是将计算机内的数据输出到外部的过程。
- ◆ 输入、输出是计算机与外界的交互手段
 - 在程序的控制下, 计算机实现输入、输出操作
 - C语言提供了一组函数来实现输入输出操作:
 - 输入函数: 通常以`get`或`scanf`开头;
 - 输出函数: 通常以`put`或`printf`开头;

◆ 本章介绍几个常用的输入输出函数：

1. 字符输入、输出函数： `getchar`、`putchar`
2. 字符串输入、输出函数： `gets`、`puts`
3. 格式化输入、输出函数： `scanf`、`printf`

3.1 字符的输入与输出

1. 函数原型

(1) 字符输入函数: `int getchar();`

功能: 接收从键盘输入的一个字符

返回值: 输入的字符 (int, 字符编码值)

(2) 字符输出函数: `int putchar(int c);`

功能: 将字符c输出

返回值: 输出的字符(int, 字符编码值, 若没有正确执行时则返回EOF)。

输出时, 把int转换成char, 即取最后一个字节的值对应的字符

2. 基本调用方式:

设: `char c;`

则有 `c = getchar();`

或 `putchar(c);`



一般, 将getchar获得的字符赋给一个字符变量待用。



用字符变量调用putchar

例3.1 使用putchar函数打印 “HUST” 字样。

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    char c1, c2, c3, c4;
```

```
    c1 = 'H';  c2 = 'U';  c3 = 'S';  c4 = 'T';
```

```
    putchar(c1);
```

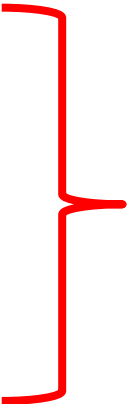
```
    putchar(c2);
```

```
    putchar(c3);
```

```
    putchar(c4);
```

```
    return 0;
```

```
}
```



连续输出四个字符变量

例3.2 可以用不同的方式表示字符的值并输出

如，设有 `char c = 'a'`，则以下方式均可输出字符'a' 的图形符号。

- `putchar('a');` //参数直接为字符常量'a'
- `putchar(c);` //参数为值为'a'的字符变量
- `putchar(97) ;` //参数为'a'的ASCII码值
- `putchar(' \141 ');` //参数为'a'的八进制转义字符表示

例3.3 调用putchar时，参数可以是复杂的表达式。

如，设有：int i;

先计算表达式的
值，然后再输出

- `putchar(i = ' ');` //参数为赋值表达式，输出一个空格
- `putchar(i = 32);` //输出一个空格
- `putchar( i ? 'Y': 'N' );` // i≠0时输出字符Y， i = 0时输出字符N
- `putchar('\a');` //转义字符，系统扬声器响铃一次
- `putchar(i = 0x2061);` //输出字符'a '
- `putchar(i = -1);` //输出？

例3.4 getchar函数的调用和执行流程分析。

```
#include <stdio.h>
```

```
int main(void)
```

```
{ char ch1, ch2, ch3;
```

```
    ch1=getchar(); //注：赋值转换
```

```
    ch2=getchar();
```

```
    ch3=getchar();
```

```
    printf("\n%c%c%c", ch1, ch2, ch3);
```

```
    printf("\n%d %d %d", ch1, ch2, ch3);
```

```
    return 0;
```

```
}
```

设输入：

a ↙

b ↙

换行符\n

则输出：

a ↙

b ↙

c呢?

97 10 98

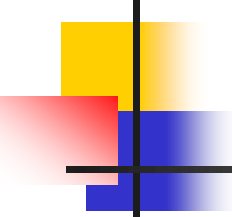
事实上还是3个字符，
ch2的值是10

- ◆ 计算机中由操作系统维护着一个称之为“**输入流**”的内存区域，从键盘输入的字符首先存储在输入流中等待输入函数的读取。

如：假定输入：abc ✓

a	b	c	\n		
---	---	---	----	--	--

- ◆ **getchar函数的执行流程**：首先检查**输入流**中是否有字符，
 - 若有则读取输入流中的第一个字符作为函数返回值，程序继续执行。
 - 若输入流中没有字符，getchar函数将进入等待输入状态。
 - 按下回车键同时会激活处于等待输入状态的getchar函数，然后读取最新输入的字符。



如:

```
ch1=getchar();
```

```
ch2=getchar();
```

```
ch3=getchar();
```

输入:

a ↵

b ↵

输入流:

a	\n				
---	----	--	--	--	--

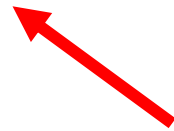
a赋给ch1

b	\n				
---	----	--	--	--	--

\n赋给ch2

\n					
----	--	--	--	--	--

b赋给ch3

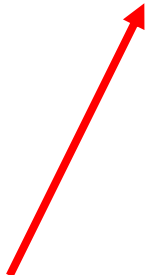


最后一个\n留在输入流中

3.2 字符串的输入与输出

◆ 字符串：0~多个字符的序列

- 字符串常量: "C Language"
- 字符串变量: `char name[10]="Wang";`



w	a	n	g	\0	
---	---	---	---	----	--

以空字符'\0'
作为结束标志

- C语言用**字符数组**存放字符串
- 数组名代表数组（字符串）在**内存中的位置**
- 通过这个“位置”就可访问数组（字符串）

1. 函数原型

(1) 字符串输出函数: `int puts(const char *s);`

参数s类型为**常量字符指针**,
代表字符串在内存中的位置

s中的内容不可更改

功能: 将字符串s输出

返回值: int, 输出的字符数,

正确执行时返回一个非负整数值, 如果出错, 则返回EOF。

(2) 字符串输入函数: `char * gets(char *s);`

参数s类型为**字符指针**

代表字符串在内存中的位置

内容能被更改

功能: 接收一个字符串存入s中

返回值: char *, 值为s的首地址。

gets从输入流中的第一个字符开始读取一个字符序列, 直到碰到**换行符**\n为止, 然后将\n替换成**字符串结束标志**\0, 最后将该字符序列 (包括\0) 存入s指定的内存缓冲区

2. 基本调用形式

设有: `char s[5] = "abc";`

`puts(s);` 以字符数组s调用puts, 将s中的内容输出到屏幕上

`puts("abc");` 以字符串常量调用, 将 "abc" 输出到屏幕上

`gets(s);` 以字符数组s调用gets, 将键盘输入的字符串存入s中

注: 以字符数组调用gets时, 应保证字符数组的大小足够存放输入的字符串, 如果长度不够就会发生溢出

a	b	c	\0	
---	---	---	----	--

输入 "abc", 正常

a	b	c	d	\0
---	---	---	---	----

输入 "abcd", 正常

a	b	c	d	e	\0
---	---	---	---	---	----

输入 "abcde", 溢出

`char s[5]`

例3.6 使用puts函数在第一行输出"Hello," , 在第二行输出"World!"。

程序1:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
char s[20] = "Hello," ;
```

```
char *pc = "World!" ;
```

```
puts(s);
```

```
puts(pc);
```

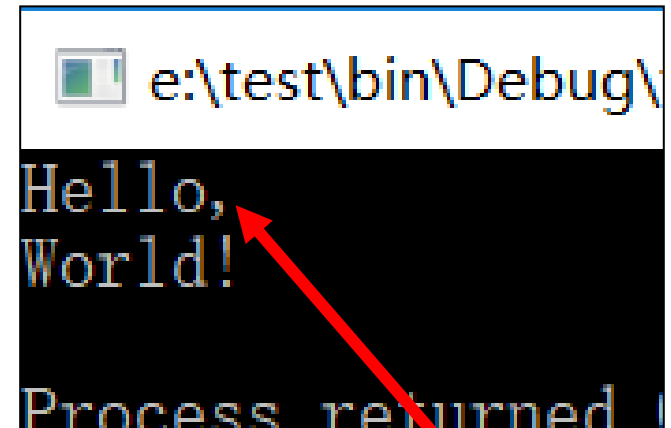
```
return 0;
```

```
}
```

字符数组

字符指针

输出:



输出时puts在每行的最后自动加了换行符

程序2:

```
#include <stdio.h>
```

转义字符

```
int main(void)
```

```
{
```

```
puts( "Hello,\nWorld!" );
```

```
return 0;
```

```
}
```

字符串常量

```
e:\test\bin\Debug\test.exe
Hello,
World!

Process returned 0 (0x0)   execution time : 0.282 s
Press any key to continue.
```


3.3 格式化输入与输出

格式化：按照指定的样式输入、输出。

1. 格式化输出函数printf

(1)函数原型： `int printf(const char *format, ...);`

- **format**是一个称为**格式字符串**的字符串，用来规定输出的格式；
- **“...”** 是0个到多个**其余的参数**，是要被输出的数据。
- **功能**：按照**格式字符串**format规定的格式，将格式串中的固定内容和后面的参数输出到标准输出设备上。
- **返回值**：int，值是函数调用时实际输出的字符个数。

```
如：char name[10] = "Wang";  
     printf("My name is %s.\n", name);
```

如: `printf("My name is %s.\n", name);`

(2) 调用形式

必需, 不能没有

`printf(格式字符串, 数据项1, ..., 数据项n)`

待输出的数据, 可以没有

- 数据项可以是0个或任意多个, 其数目、数据类型和顺序应与格式字符串中的格式转换说明一致;
 - 如果不一致, 编译时通常不会报错, 但是不能得到正确的输出结果。
 - 数据项可以是表达式, 计值后输出其结果;

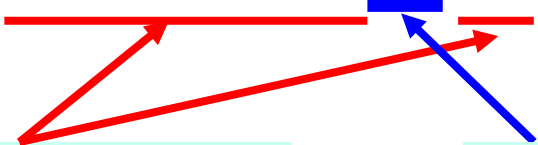
(3) printf函数格式字符串的组成

包含两类字符：

- **普通字符**：按原样输出。
- **格式字符**：以%字符开始，以转换字符结尾。

输出时被参数代换。

```
printf("My name is %s.\n", name);
```



普通字符（包括空格
和标点符号，但不包
括两边的双引号）

格式字符

(4) 格式字符的语法形式为：

%[域宽说明]转换字符

域宽说明：用来指出数据输出时的**对齐方向**，**输出数据域的宽度**、**小数部分的位数**等格式要求。

转换字符：用来指出所输出**数据的类型**和**样式**。

重点
关注

%d：输出整数、
%c：输出字符、
%s：输出字符串、
%f：输出浮点数

作用：占位符，执行时，用后面的数据项进行替换

例：设变量说明为：

char c=65;

int x1=65;

unsigned x2=65535; float y=65;

char name[100]= "The C programming" ;

则

printf("%c,%d,%d,%c,%u,%f,%s" ,c, c, x1, x1, x2, y, name);

输出：

A,65,65,A,65535,65.000000,The C programming

默认情况下，浮点数输出精度为小数点后6位。

域宽说明

- ◆ 在%和转换字符之间可以有域宽说明字符。域宽说明用来指出输出数据的对齐方式、输出数据域的宽度、小数部分的位数等要求。【可用作域宽说明的字符见表3.2，域宽说明可以是其中一个或多个字符的组合】
- ◆ 常用的域宽说明字符：
 - : 输出时，数据左对齐
 - .n : 整数最少位数、或浮点数小数位数、或字符串最多字符数
 - m : 整数最少域宽、浮点数最少位数、或字符串最少字符数
 - h : 短整数
 - l : 长整数
 - L : 长双精度浮点数

例3.9 输出整数时，-和m.n的用法

设： int i1=12, i2=12345;

(1) printf("i1=%-8.3d,i1=%8.3d\n" , i1, i1);

输出： i1=012 ,i1=012

左对齐

8位域宽，不足时后补空格

最少3个数码，不足的补前导0

右对齐

8位域宽，不足时前补空格

最少3个数码，不足的补前导0

(2) printf("i2=%-8.3d,i2=%8.3d\n" , i2, i2);

输出： i2=12345 ,i2=12345

左对齐。8位域宽，最少3个

数码，超出3位原样输出

右对齐。8位域宽，最少3个

数码，超出3位原样输出

输出浮点数，-和m.n的用法

设： float x=3.1416; double y=123.4567;

(3) printf("y=%4.2f\n" , y);

输出： y=123.46

最小4位域宽（最少输出4个字符，包括小数点），这里实际6位。
精度：小数点后2位，多的四舍五入

(4) printf("x=%8.3f,x=%-.3f\n" , x, x);

输出： x= 3.142,x=3.142

最小8位域宽，不足补空格。
精度：小数点后3位，多的四舍五入
默认为右对齐

没有最小域宽要求。精度为小数点后3位，多的四舍五入。
左对齐

输出字符串时，-和m.n的用法

```
(5) printf( "%8s,%-8.4s,%8.4s\n" , s1, s1);
```

输出: Technology,Tech , Tech

最小8位域宽,
超出原样输出

左对齐, 8位域宽, 但只
输出4位, 少的部分补后
导空格

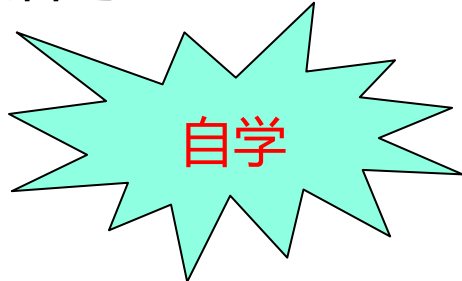
右对齐, 8位域宽, 也只
输出4位, 少的部分补前
导空格

例：设变量说明为int i = 15; float x = -8.2;; 则语句

```
printf("i=%d, x=%8.3f\n", i, x);
```

执行时输出：

i = 15, x = -8.200 (-8.200左边有两个空格)



自学

说明：格式字符串 “i=%d, x=%8.3f\n” 中，

- “i=” 、 “,” 、 “x=” 和 “\n” 是普通字符;
- %d和%8.3f是两个转换说明，d和f是转换字符；8.3是域宽说明。
- 转换说明%d对应于第二个参数i，%8.3f对应于第三个参数x；输出时格式字符串中的%d和%8.3f分别被i和x的值所代替。

(5) 转换说明与输出参数不一致会怎么样？

- printf函数的格式字符串中的转换说明应和输出数据项的数目、类型和顺序一致；
- 如果转换字符使用不当，或转换说明的个数多于数据项个数，则会输出错误的数据；
 - 关于输出类型不匹配可能引发的输出问题，请参见：
 - <https://www.zhihu.com/question/29300881>
- 如果转换说明的个数少于数据项个数，则多出的数据项不被输出。

例如：设 `int i=-6; double x=5.7, y=123.4567;`

(1) `printf( "%d,%g\n" , i, y, x );`

输出： `-6,123.457`

数据项多了，x因无对应的转换说明，未被输出。

(2) `printf( "%d\n" , x );`

输出： `-858993459`

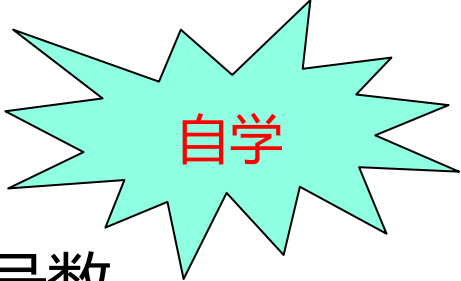
x的类型与转换说明%d不匹配。将x的低32位按照整数进行解析并输出。

(3) `printf( "i=%d\n" );`

输出： `i=4109`

参数里缺少数据项，随机输出一个不确定的值4109

(6) 输出无符号数： %u



自学

◆ %d用于输出有符号数、%u用于输出无符号数。

➤ 二进制最高位为0的整数：两者输出相同的结果(都是正值)。

如：设 `int i = 100; unsigned j = 100;`

```
printf( "%d,%u" , i, i);          //输出100,100
```

```
printf( "%d,%u" , j, j);          //输出100,100
```

➤ 二进制最高位为1的整数：%d输出负值，%u输出的正值。

如：设 `int i = ~0; unsigned j = ~0;`

i和j的字节内容是一样的

```
printf( "%d,%u" , i, i);          //输出-1,65535
```

```
printf( "%d,%u" , j, j);          //输出-1,65535
```

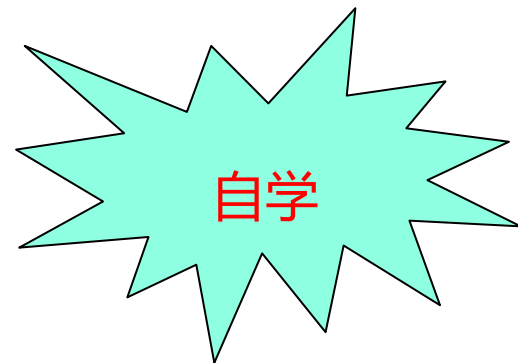
转换字符

转换字符	参数类型	输出格式
d , i	int	十进制整数
o	int	八进制整数（不带前缀0）
x , X	int	十六进制数（不带前缀0x或0X）
u	int	无符号十进制数
c	char	单个字符
s	char *	字符串（必须以' \0' 结束或在域宽说明中给出长度限制）
f	double	小数形式的浮点数（小数部分位数由精度确定，缺省为6位）
e, E	double	标准指数形式的浮点数（尾数部分位数由精度确定）
g, G	double	在不输出无效零的前提下，按输出域宽度较小的原则从%f和%e两种格式中自动选择
p	void *	指针值（输出格式与具体实现有关）
n	int *	将到达该转换说明为止已输出的字符数目写入对应的参数中，不转换参数
%	不转换参数	输出一个%字符



关于printf的格式字符串，还有以下内容：

- ◆ (7) 浮点数的其他输出形式：%e, %g
- ◆ (8) 特殊转换字符n、%的用法
- ◆ (9) h, l, L的用法
- ◆ (10) 可变域宽
- ◆ 以及八进制、十六进制数的输出、指针型数据的输出等



请阅读课件后面的内容及教材的相关内容，自学。

(7) 浮点数的其他输出形式: %e, %g

- ◆ %f、%e、%g区别在于缺省域宽说明时的输出精度
 - %f输出小数形式的浮点数，小数部分为6位。
 - 多于6位采用四舍五入，少于6位则在末位补零，以保证6位小数；
 - %e输出标准指数形式的浮点数，尾数部分为6个有效数字。
 - 包括1位非零的整数部分和5位小数部分，采用四舍五入和末位补零的方法确保6个有效数字；
 - %g将按%f和%e两种格式输出的数据去掉无效零后进行比较，选取输出宽度较小的那种格式进行输出。

例：设变量说明为

```
double x=12.00004951, y=12.4951, z=12000000.4951;
```

则

```
printf( "%f\t%e\t%g\n%f\t%e\t%g\n" , x, x, x, y, y, y);
```

```
printf( "%f\t%e\t%g\n" , z, z, z);
```

输出：

12.000050	1.20000e+01	12
12.495100	1.24951e+01	12.4951
12000000.495100	1.20000e+07	1.2e+07

(8) 特殊转换字符n、 %的用法

例3.7 转换字符n的用法。

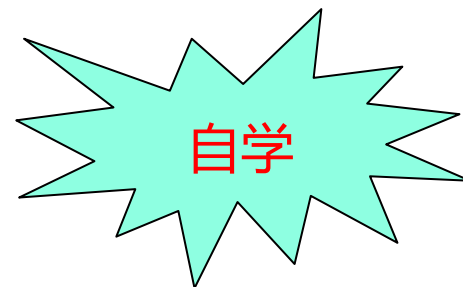
设有：int x = 1;; 依次执行下面两个语句：

```
printf( "abc%n" , &x);
```

```
printf( "x=%d" , x);
```

输出：abc

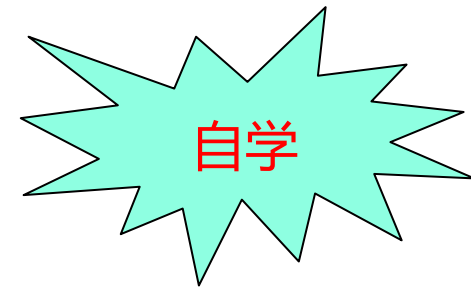
3



分析：第一个语句首先输出%n之前的字符abc，然后统计到%n为止已输出的字符数目（这里是3），最后该字符数目写到按对应参数所表示的内存地址中去（这里&x表示变量x的地址），所以x的值被改写为3；

第二个语句输出整数x的当前值，即3。

例3.8 转换字符%的用法。



情况1: `printf( "%%d" , 100);`

输出1: `%d`

分析1: 相邻的两个%输出一个%, 字符d作为普通字符输出。
而后面的数据项100不被输出。

情况2: `printf( "%%%d" , 100);`

输出2: `%100`

分析2: 相邻的两个%输出一个%, %%后面的%d是输出十进制
整数转换说明, 所以继续输出后面的数据项100。

特别说明:

自学

- ◆ 如果%后面的字符是一个%, 则输出一个%;
- ◆ 如果%与其后的字符不能构成一个合法的转换说明, 大多数编译系统将%与其后面的字符一起作为普通字符输出。

例如: 设 `int i=-6; double x=5.7, y=123.4567;`

(1) `printf( "%d%%100" , -i/2);`

输出: **3%100**

- 相邻的两个%输出一个%
- 100原样输出。

(2) `printf( "%a" , i);`

输出: **%a**

- 因为%后面的a不是转换字符, %a被当作普通字符输出。
- 由于格式字符串中没有转换说明, 所以后面的数据项i不被输出。

(9) h, l, L的用法

自学

例3.10 设: short a; long b;
double x; long double y;

```
printf("a=%hd,b=%ld,x=%lf,y=%Lf", a, b, x, y);
```

输出短整数

输出长整数

输出double类型的浮点数

输出long double类型的浮点数

(10) 可变域宽

- ◆ *****: 可变域宽, 代表一个整数, 其值由对应的参数决定, 可用于替代m和n表示的可变域宽或精度。

例3.11 : 设 `int max=6; char s[10]="123456789";`

(1) `printf( "%*c" , max, ' ' );`

输出: 6个空格

*代表的域宽, 由对应参数max决定

(2) `printf( "%*s" , 2*max, s);`

输出: 123456789

*代表的域宽由对应参数2*max决定(12)

左边填充了3个前导空格

(3) `printf( "%.*s" , max, s);`

输出: 123456

*代表的输出精度, 由对应参数max决定

2. 格式化输入函数scanf

(1) 函数原型:

```
int scanf (const char *format, ...);
```

- **format**: 格式字符串, 用来指定输入数据的数目、类型、格式、顺序;
- **, ...**: 可变数目的其他参数, 用来接收其输入的数据。
- **返回值**: int, 执行正确时返回实际接收的数据的个数, 遇到文件尾或执行出错时返回EOF。

(2) scanf函数的格式字符串

scanf函数的格式字符串与printf函数相似，由**转换说明**和**普通字符**两部分组成：

(1) **转换说明**：以**%**开头，以**转换字符**结尾

- 基本形式：**%转换字符**
- 常用的scanf函数转换字符如P63表3.3所示

(2) **普通字符**：指转换说明之外的其它字符

又细分为**空白字符**和**非空白字符**。

- ◆ **scanf函数的格式字符串最常见形式是只包含转换说明;**

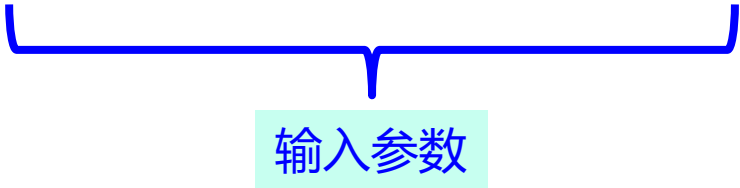
- **重点掌握的转换说明:**

- **%d:** 输入整数
 - **%c:** 输入字符
 - **%f:** 输入浮点数
 - **%s:** 输入字符串

- ◆ **关于普通字符的问题见后**

(3) scanf函数的输入参数1、输入参数2...

`scanf` (格式字符串, 输入参数1, 输入参数2, ..., 输入参数n);



输入参数

- 除格式字符串外，至少应有一个输入参数。
- 输入参数必须是地址型的量，可以是基本类型变量的地址或指针类型变量。
- 输入参数在类型、数目和次序上应与格式字符串中的转换说明一致。

(4) 调用形式

`scanf` (格式字符串, 输入参数1, 输入参数2, ..., 输入参数n);

作用：从标准输入设备（输入流）中读取字符流，并按照**格式字符串**中由**转换字符**规定的**格式转换**成相应类型的数据后，**存放到输入参数指定的内存单元**中。

如：int x; char c;



`scanf("%d%c", &x, &c);`

这是什么意思？

形式上同于printf的格式字符串

如何改变内存单元的内容？

如何获取变量的地址?

含义：获取输入的整数后，将整数的值存放到由地址&x标识的内存单元中，简称为存入x单元。

(a) 取地址运算：&

如：int x;

scanf("%d",&x);

(b) 数组名就代表地址

如：char name[10];

scanf("%s",name);

scanf函数需要以变量的地址或数组名作为参数，接收输入

含义：将获取的字符串存放到name所标识的内存单元中。

例3.12 设变量说明为

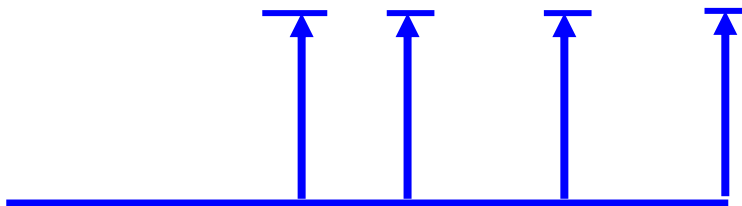
```
char c;  
int x1;  
unsigned x2;  
float y;  
char name[10];
```

字符数组名本身就是地址类型，不需要再用&取地址

则输入以上类型数据的语句为：

```
scanf( "%c%d%u%f%s" , &c, &x1, &x2, &y, name);
```

取地址运算



(5) 转换说明与输入参数不一致时可能导致的后果

如果转换说明与输入参数的类型不匹配，会导致读入的数据值不正确或程序非正常终止；

例如： 设 `int i, j;`

`double x;`

执行： `scanf( "%d%d" , &i, &j);`

输入： `12a` ✓

结果： `i=12, j未赋值, 函数返回1`

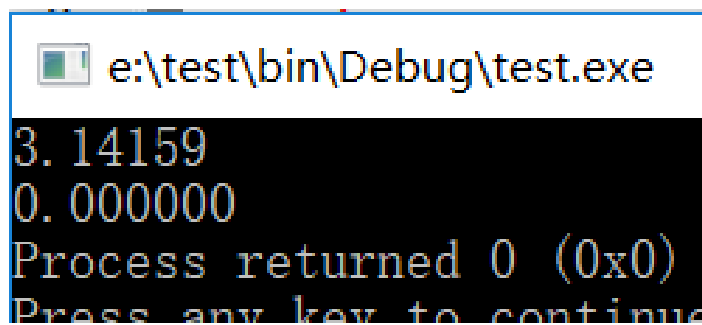
分析：整数12被赋予i，而由于第二个输入域是字符a，与第二个转换说明%d不匹配，因而不能被转换，j未被赋值；实际正确数据个数是1，所以scanf返回1.

再如：

- `scanf( "%d%d" , &i);`

分析：这里有两个转换说明，但只有一个输入参数，此种情况可能会造成系统崩溃而死机

- `scanf( "%f" , &x);`
`printf("%f",x);`



```
e:\test\bin\Debug\test.exe
3.14159
0.000000
Process returned 0 (0x0)
Press any key to continue...
```

分析：%f对应输入的是float型浮点数，而x是double型，两者类型不匹配，实际输入时x的值不是预期的值。

- 正确的方式：`scanf( "%lf" , &x);` 或将x声明为float型

(6) 输入形式和要求

scanf对格式字符串的每个转换说明确定一个输入域，然后根据转换说明对输入域内的字符序列进行数据转换。

- ◆ 空白字符：特指空格符、换行符、制表符等。
- ◆ 输入域：从输入流中当前位置开始，直到遇到第一个
 - (1) 空白字符，或
 - (2) 根据转换说明不能被转换的字符，或
 - (3) 达到了指定的域宽，至此范围内的所有字符构成一个输入域。

输入域的格式要求:

- (a) 输入域之间一般用**空白字符**隔开; 但是
- (b) 整型、浮点型或字符型数据与**后面的字符型数据**之间不能用空白符隔开, 否则该空白符将被作为下一个字符读入;

例3.14 设有: `int i; char c;`

```
scanf("%i%c",&i, &c);
```

(1) 若输入: **1234a**, 则, 1234赋给i, a赋给c, **正确!**

(2) 若输入: **1234 a**, 则, 1234赋给i, **空格**赋给c, **不正确!**

另, 自学例3.15、例3.16

(c) 整型、浮点型或字符型数据与后面的字符串数据之间可以有或也可以没有空白字符;

例3.17 设有:

```
int day, year;
```

```
char month[10];
```

```
scanf("%d%s%d", &day, month, &year);
```

若输入: 25 Dec 1996 或 25Dec 1996

则, 25赋给day, "Dec" 赋给month, 1996赋给year
均正确!

(d) 一个字符串内部不能有空白字符，因为空白字符是输入域的分隔符。没有被空白符隔开的字符串被视为一个整体。

例 设有：

```
int day, year;  
char month[10];  
scanf("%d%s%d", &day, month, &year);
```

若输入： 25 Dec1996

则， 25赋给day， "Dec1996" 赋给month， year没被赋值
不正确！

另，自学例3.18

(7) 数据转换的过程

- ◆ 当输入完转换说明规定数目的数据并键入回车键时scanf函数开始执行;
- ◆ 当用完格式字符串中的转换说明或当某个输入域与转换说明不能匹配时, scanf停止执行;

例3.19 设有

```
int i, j;    float x, y;
```

```
scanf("%d%d",&i,&j,&x);
```

```
scanf("%f",&y);
```

这里, 输入数据项多于转换说明

如果上一次调用scanf, 输入流中待转换的数据没有用完, 则下一次调用时接着读取。

输入: 12 34 56.7 89

结果: 12 赋给i

34 赋给j

x没有被赋值

56.7 赋给y

89 没有使用到

(8) 当scanf函数的格式字符串中包含有非空白字符的普通字符时，怎么处理？

处理方式：必须在输入流中的相应位置输入相同的字符与之一一对应。

例3.20 设有 int i, j, k;

```
scanf("%d,%d,%d",&i,&j,&k);
```

正确输入： 10,20,30

或： 10, 或： 10
 20, ,20,30
 30

错误输入： 10 20,30

或： 10,,20,30

或： 10,20,:30

例3.21 设有 int i, j;

```
scanf("i=%d;j=%d;%d", &i, &j);
```

正确输入: i=10; j=20

或: i=10;

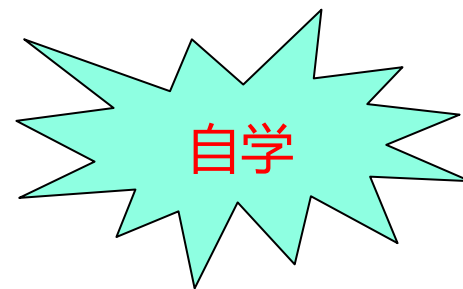
j=20

格式字符串含有非空白字符的普通字符 “i=” 、 “j=” 及分号符 “;”, 所以执行时必须在对应的位置照样输入这些字符。

如果需要输入一个带空白字符的字符串，怎么办？

如：My name is Wang Duoqiang.

- 必须使用gets函数来实现
 - 而不能用scanf直接输入。
-
- 需要说明的是，标准函数gets在C语言标准ISO-IEC 9899:2011（简称C11）之前的版本中是可以使用的。由于该函数隐含了编译器无法判断的安全隐患，因此在C11中删除了该函数（取而代之的gets_s）。考虑到C11标准的编译器还未进入应用阶段，这里仍给出了该函数的介绍。



(9) 其它转换字符

(a) **n**: 统计前面成功处理过的字符数

例3.13 设有: `int i, j, k;`

`scanf("%d%d%n", &i, &j, &k)`

输入: 10 20 30 ✓

结果: 10被赋给*i*, 20被赋给*j*, 6赋给了*k*。

分析: 对应第三个转换说明%n, 不从输入流中读取数据30, 而是把到此为止已从输入流中读过的**字符数目6**赋给*k* (包括10和20之间的一个空格字符)。

最后输入流中还剩余字符 ' ', '3', '0'和'\n'四个字符。

(b) %: 如果转换说明为%%, 则不从输入流读取数据, 对应参数的值也不改变;

例如: 设有 `int i=1;`

```
scanf("%%d", &i);
```

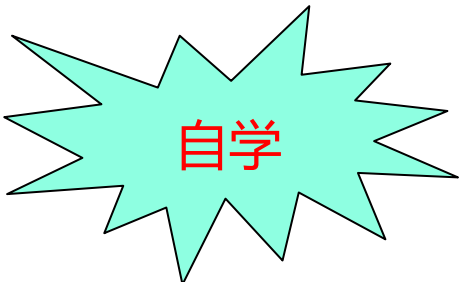
```
printf("i=%d", i);
```

输入: 100 ✓ (或%100 ✓)

输出: i=1

- 如果在其他完整的转换说明前面有两个相邻的%, 则输入时需要一个%与之匹配。

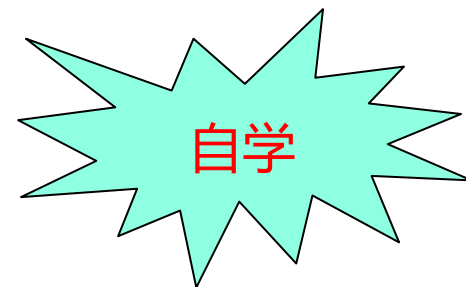
如: 上例可改成`scanf("%%%d", &i);`



自学

(10) scanf函数格式字符串中转换说明的一般形式

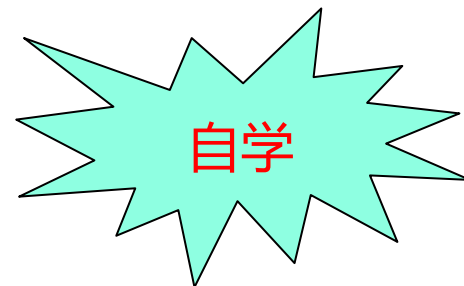
- ◆ 最常用的是基本形式：**%转换字符**
- ◆ 转换说明的一般形式：**%[选项]转换字符**
 - 常用的scanf函数转换字符如P63表3.3所示



◆ 选项包括：

- **域宽**：从自然输入域首字符开始取指定宽度的字符作为**实际输入域**。
- **h, l和L**：转换修饰字符，用于输入短整数(%hd)、长整数(%ld)、双精度浮点数(%lf)、长双精度浮点数(%Lf)
- **虚读**：在转换字符前面使用 “*” (%*d)，赋值抑制符

(a) **域宽**：如果指定了域宽，则从自然输入域首字符开始取指定宽度的字符作为实际输入域。



例 3.22 设有 `int i, j;`

```
scanf("%3d %d", &i, &j);
```

输入：1234 5678

结果：i=123

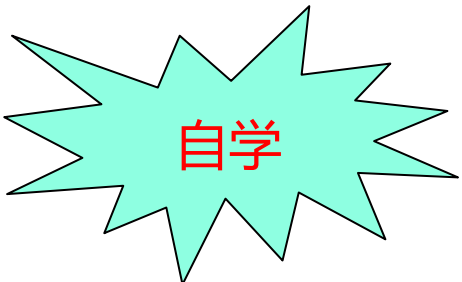
j=4

5678保留在输入流中

另见例3.23

- 若自然输入域的宽度小于或等于指定的域宽，整个自然输入域将作为实际输入域；
- 若自然输入域的宽度大于指定域宽，则从自然输入域取完指定宽度的字符后，剩下的字符作为下一个自然输入域。

(b) **h**, **l**和**L**: 输入短整数(**%hd**)、长整数(**%ld**)、双精度浮点数(**%lf**)、长双精度浮点数(**%llf**)。



自学

例 3.24 设有 short i;

long j;

double x;

long double y;

scanf("%hd%ld%lf%Lf", &i, &j, &x, &y);

(c) **抑制赋值符'**

——这种情况称为“**虚读**”。

自学

- 虚读用于从输入流中有选择地读取部分内容。

例3.25：设有 `int n1,n2,n3;`

```
scanf( "%d %*d %d" ,&n1,&n2,&n3);
```

输入： **11** **22** **33** **44**

结果： n1、 n2、 n3的值分别是： **11** **33** **44**



【另见例3.26】

22被虚读，导致33被赋给n2

可变形参:

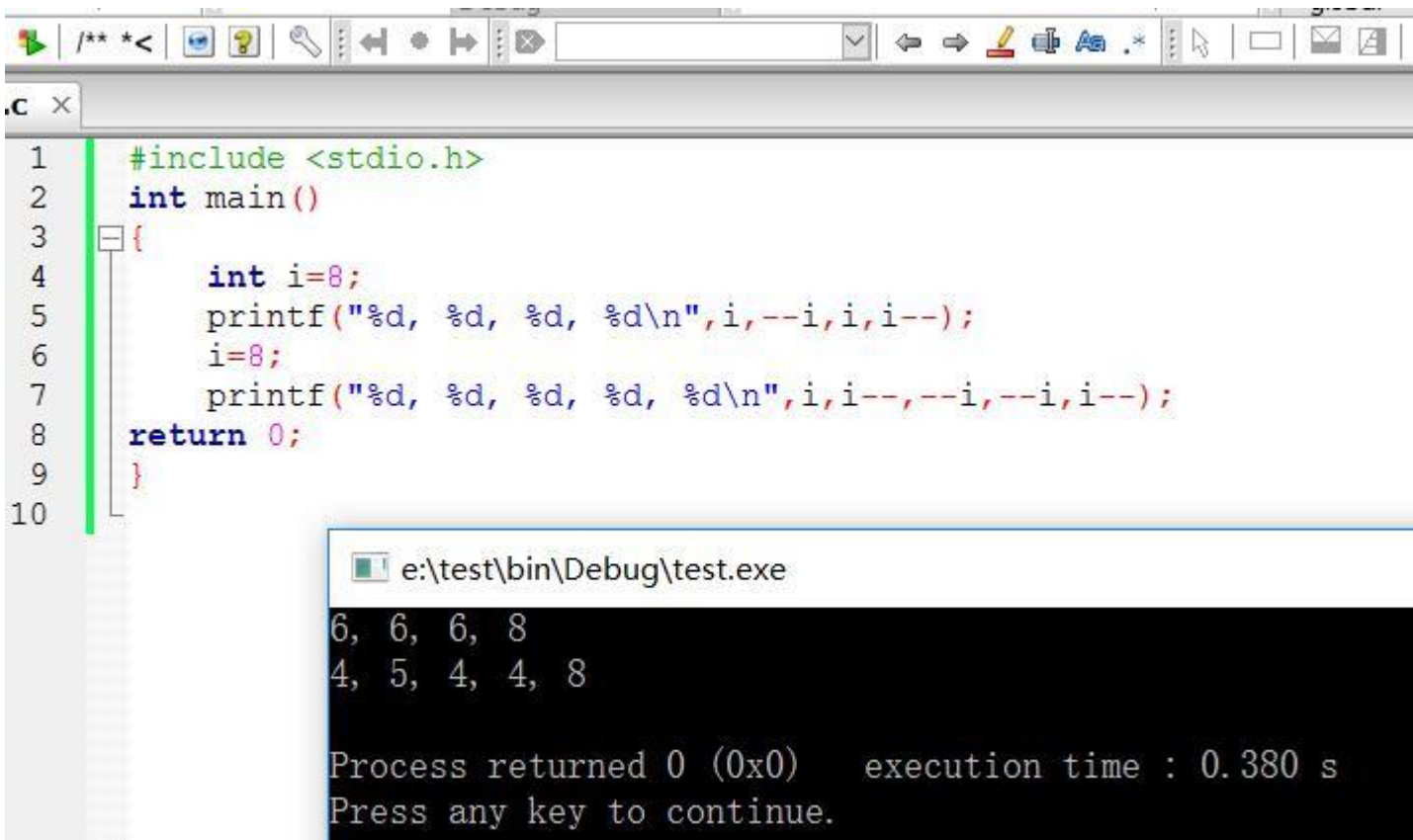
```
int printf(const char *format, ...);
```

```
int scanf (const char *format, ...);
```

其中, “,...” 是可变参数的表示形式, 表示该函数在调用时除前面的字符串参数必需之外, 其余参数的数目可变, 可以是0个到多个。

➤ 具体参见函数的内容

■ 特殊现象



```
1  #include <stdio.h>
2  int main()
3  {
4      int i=8;
5      printf("%d, %d, %d, %d\n", i, --i, i, i--);
6      i=8;
7      printf("%d, %d, %d, %d, %d\n", i, i--, --i, --i, i--);
8      return 0;
9  }
10
```

e:\test\bin\Debug\test.exe

6, 6, 6, 8
4, 5, 4, 4, 8

Process returned 0 (0x0) execution time : 0.380 s
Press any key to continue.

本章小结

本章介绍了几个基本的标准输入与输出函数，包括

- 单个字符输入输出函数 `getchar` 与 `putchar`
- 字符串输入输出函数 `gets` 和 `puts`,
- 格式化输入输出函数 `scanf` 和 `printf`
- ◆ 在编程时，这些函数的使用频度非常高，因此需要理解和掌握其用法，在编程时做到灵活应用。
- ◆ 格式化输入输出中常用转换说明 `%c`、`%d`、`%s`、`%f` 等的用法需要记住。